PROJECT EXECUTION BLUEPRINT

BehaviorIQ — A Real-Time, LLM-Augmented Behavioral Intelligence Platform

Real-time clickstream capture • Transformer-based session modeling • Vector retrieval • Contextual-bandit personalization • LLM natural-language analytics copilot • Full MLOps

---

A single, continuous build plan — no semesters, no fluff, just the complete system, stage by stage, with the exact architecture, tools and evaluation criteria for each part.

Includes a novel, publication-grade research paper direction at the end.

Table of Contents

- 1. Vision — What Makes This a Top-Tier System
- 2. The Problem
- 3. System Overview — The Full Architecture
- 4. Complete Technology Stack (Core + Differentiator Layers)
- 5. Data Design, Event Schema & Privacy Engineering
- 6. Execution Plan — 13 Build Stages, Fully Detailed
- 7. Evaluation Framework & Success Bar for Every Component
- 8. The Research Paper
- 9. Execution Checklist (Definition of Done, Master List)

1. Vision — What Makes This a Top-Tier System

Most "behavior analytics" student projects stop at: collect clicks, train one classifier, put numbers on a dashboard. We are deliberately building past that ceiling. The system you are building, **BehaviorIQ**, is designed to look and function like something a modern data/ML platform team would actually ship — because it borrows the same architecture patterns real companies use in 2026: streaming-first ingestion, a feature store, transformer-based sequence representation learning, a production vector database, retrieval-augmented prediction, contextual-bandit personalization, and — the single biggest differentiator — an **LLM-powered natural-language analytics copilot** that lets anyone ask the platform questions in plain English and get grounded, data-backed answers.

The one-line pitch: BehaviorIQ watches how users move through a product in real time, represents each session as a learned behavioral "fingerprint," retrieves and predicts intent from that fingerprint, personalizes decisions on the fly, and lets a human simply *ask* the platform what's happening — no SQL, no dashboards required.

Why this specific combination is the right one to build right now

- **Real-time streaming architecture** (Kafka/Redpanda + Flink) is the backbone of every serious data platform today — payments, ads, fraud, recommendations. Demonstrating you can build one, correctly, matters more than any single model.
- **Transformer-based sequence modeling** (the SASRec/BERT4Rec family) is the actual state of the art in industrial recommendation and intent systems — far past "logistic regression on session stats."
- **Vector databases and retrieval-augmented prediction** are now core infrastructure, not a research curiosity — every modern personalization and search stack has one.
- **Contextual bandits** are how real platforms make personalization decisions under uncertainty instead of static rule-based targeting — this is what separates "we built a classifier" from "we built a decisioning system."
- **An LLM analytics copilot with retrieval-augmented generation (RAG)** is the single most in-demand capability in the current market. Being able to say "I built an agent that answers business questions over a live behavioral warehouse, grounded and hallucination-checked" is an immediate attention-grabber.
- **Full MLOps** (experiment tracking, model registry, drift monitoring, CI/CD, observability) proves the system isn't a notebook — it's operable, monitorable, and re-trainable, which is what production teams actually care about.

Every one of these pieces is achievable by a small student team if you build in the order given in Section 6 — none of it requires infrastructure you cannot run on a laptop plus free-tier cloud credits (AWS/GCP/Azure student credits, or entirely local via Docker).

2. The Problem

Conventional analytics tools answer *what happened, in aggregate*. They cannot tell you the difference between two users who spent the identical 4 minutes on a product but had opposite intent:

User A (converts)	User B (abandons)
Search → Product → Add to Cart → Purchase	Search → Product → Reviews → Compare → Product → Reviews → Add to Cart → Remove → Compare → Exit

Core problem statement: How can raw, streaming user-interaction events be transformed, in real time, into learned behavioral representations that support (a) similarity retrieval across millions of past sessions, (b) accurate intent prediction as a session unfolds, (c) automated personalization decisions, and (d) natural-language business querying — without hand-engineering features for every new question a stakeholder might ask?

Four capabilities are demanded by that problem statement, and each maps directly onto a subsystem you will build:

Capability	Subsystem
Understand a session as it happens	Real-time streaming ingestion + feature store
Compare a session against history	Transformer session encoder + vector database
Predict what happens next	Retrieval-augmented intent model + contextual bandit
Let a human just ask	LLM analytics copilot (RAG + text-to-SQL agent)

3. System Overview — The Full Architecture

```
USER (browser / demo storefront)
  |
  v
TRACKING SDK  --(events, batched)-->  INGESTION API (FastAPI)
  |
  v
STREAMING BACKBONE (Kafka / Redpanda)
  |
  +---> REAL-TIME STREAM PROCESSOR (Apache Flink)
  |         -> session windowing, live feature computation
  |         -> writes to FEATURE STORE (Feast: Redis online + Parquet offline)
  |
  +---> BATCH ETL (Apache Spark)
            -> cleans, dedups, backfills
            -> writes to WAREHOUSE (ClickHouse: real-time OLAP + Postgres: dims/config)

WAREHOUSE + FEATURE STORE
  |
  +---> BEHAVIORAL MINING  (PrefixSpan, FP-Growth, HDBSCAN clustering)
  |
  +---> SEQUENCE REPRESENTATION LEARNING
  |        -> Transformer session encoder (SASRec-style), self-supervised
  |           contrastive pretraining on sequences
  |        -> session embeddings
  |
  +---> VECTOR DATABASE (Qdrant)
  |        -> stores every session embedding + outcome
  |        -> powers similarity search / retrieval-augmented features
  |
  +---> INTENT PREDICTION + PERSONALIZATION
  |        -> retrieval-augmented intent classifier (XGBoost / small Transformer head)
  |        -> contextual bandit (Thompson Sampling) chooses the next best action
  |
  +---> LLM ANALYTICS COPILOT
            -> RAG over ClickHouse (text-to-SQL) + Qdrant (semantic session search)
            -> answers natural-language business questions, grounded in real data

MLOPS & OBSERVABILITY (cross-cutting)
  MLflow (experiment tracking + model registry) | Prometheus + Grafana (system/model monitoring)
  Evidently AI (drift detection) | SHAP (explainability) | GitHub Actions (CI/CD) | Docker + Kub
ernetes

PRESENTATION LAYER
  React dashboard (live behavioral segments, intent scores, semantic search)
  + embedded LLM chat panel ("ask BehaviorIQ")
  + Power BI executive view (optional, for stakeholder-style reporting)
```

This looks like a lot of moving parts — it is. But every arrow in this diagram becomes exactly one numbered stage in Section 6, built and tested in isolation before being wired to the next. You never build more than one arrow at a time.

4. Complete Technology Stack (Core + Differentiator Layers)

Tools are grouped into **Core** (the system does not function without these) and **Differentiator** (these are what make the project stand out — build them once Core is stable).

Core layer

Layer	Technology	Why
Demo storefront	Next.js (React) + TypeScript + Tailwind	Modern, fast to build, component-driven; Tailwind keeps styling quick so time goes into the platform, not CSS.
Tracking SDK	Hand-written TypeScript module	Full control over event schema and batching logic; shows you understand instrumentation, not just consuming a library.
Ingestion API	Python + FastAPI + Pydantic	Async, typed, auto-documented; industry-standard choice for high-throughput APIs.
Streaming backbone	Apache Kafka (or Redpanda for lighter local dev)	The de-facto standard for durable, ordered, high-throughput event streams.
Real-time processing	Apache Flink	True low-latency, stateful stream processing with native session-windowing — the modern choice over micro-batching for this exact use case.
Batch ETL	Apache Spark	Scales cleaning/backfilling and heavier offline feature computation.
Feature store	Feast (open source)	Guarantees the exact same feature values are used online (serving) and offline (training) — eliminates training/serving skew, a real production concern.
Online feature cache	Redis	Sub-millisecond feature lookups for real-time scoring.
Analytical warehouse	ClickHouse	Columnar, real-time OLAP engine built for exactly this kind of high-volume event analytics; used by many real clickstream platforms.
Relational store	PostgreSQL	Clean home for dimension/config data and anything transactional.
Data mining	Python: mlxtend, prefixspan, scikit-learn, HDBSCAN	Sequential pattern mining, association rules, density-based clustering.

Differentiator layer — this is what makes it impressive

Layer	Technology	Why
Sequence representation learning	Custom Transformer encoder (SASRec/BERT4Rec-style), PyTorch	State-of-the-art approach to modeling ordered behavior — a real research-grade technique, not a toy embedding.
Self-supervised pretraining	Contrastive learning objective over augmented session pairs (mask/crop/reorder)	Lets the encoder learn from unlabeled traffic before any labeled purchase/abandon outcome exists — directly mirrors how modern foundation encoders are trained.

Layer	Technology	Why
Vector database	Qdrant	Production-grade, self-hostable vector search engine with filtering, used widely in real semantic search and RAG stacks.
Retrieval-augmented prediction	Custom feature: outcomes of top-K nearest historical sessions fed into the intent model	Borrows the RAG idea and applies it to prediction instead of text generation — a genuinely novel framing for a student project.
Personalization decisioning	Contextual bandit (Thompson Sampling / LinUCB)	Real platforms don't just classify, they decide an action under uncertainty and keep learning from the outcome — this is the difference between an ML model and a decision system.
LLM analytics copilot	Claude or GPT API + RAG (LangChain/LlamaIndex or hand-rolled) + text-to-SQL over ClickHouse	The single highest-attention-grabbing component: a conversational agent that answers real business questions, grounded in your own live data.
Explainability	SHAP	Every intent prediction can be explained feature-by-feature — critical for trust and for a strong viva/demo answer to "why did the model say that?"
Experiment tracking / model registry	MLflow	Tracks every training run, hyperparameter set, and metric; promotes the best model version to "production" with one click.
Drift detection	Evidently AI	Continuously checks whether incoming traffic statistically differs from training data — proves you understand models decay in the real world.
Observability	Prometheus + Grafana	Live dashboards for pipeline health, request latency, throughput, and model performance over time.
CI/CD	GitHub Actions	Automated tests + linting + build on every push; a real engineering practice, not just "it runs on my machine."
Containerization / orchestration	Docker Compose (local) -> Kubernetes (optional cloud stretch goal)	Reproducible environment for every teammate and for the final demo; Kubernetes as a stretch goal signals cloud-native maturity.
Infra as code (stretch)	Terraform	If you deploy to AWS/GCP, define the infrastructure as versioned code rather than manual clicking.

Build order matters more than tool count. Section 6 sequences these so that Core lands first and each Differentiator is added only once the layer beneath it is verified working. Never add a Differentiator tool to look impressive if the Core layer under it is still shaky — a recruiter or examiner will find the cracks in five minutes of questions.

5. Data Design, Event Schema & Privacy Engineering

Demo storefront pages/flows to instrument

```
Homepage -> Search -> Product Listing -> Product Page -> Reviews -> Compare -> Cart -> Checkout
-> Purchase
```

Canonical event schema (send exactly this shape from the SDK)

```
{
  "event_id": "uuid-v4",
  "session_id": "s_84213",
  "anon_user_id": "u_7f2a9c",
  "event_type": "product_view",
  "timestamp": "2026-08-21T09:41:28.104Z",
  "page": "/product/17",
  "product_id": "product_17",
  "device": "desktop",
  "referrer": "search",
  "meta": {"scroll_depth": 0.42, "dwell_ms": 8300}
}
```

Event vocabulary

- page_view, search, click, scroll, hover, product_view, review_view, compare, add_to_cart, remove_from_cart, checkout_start, purchase, exit

Session construction rule (write this down once, never change it silently)

A session is the ordered list of events sharing a session_id, closed after 30 minutes of inactivity OR an explicit purchase/exit event — whichever comes first. This single definition feeds every downstream component; changing it later invalidates prior embeddings and models.

Privacy engineering — build this in from day one, not as an afterthought

- Never capture names, emails, passwords, phone numbers, or free-text fields that could contain PII (sanitize/strip search queries before storage).
- All identifiers (session_id, anon_user_id) are randomly generated, never derived from real identity.
- Apply **k-anonymity** style aggregation before any figure leaves the warehouse for the LLM copilot or dashboard — never surface a metric derived from fewer than a set minimum number of sessions.
- Optional stretch: add **differential privacy noise** (via the Opacus or diffprivlib library) to aggregate query results returned by the LLM copilot, so even aggregate answers carry a formal privacy guarantee — a strong, unusual, and genuinely impressive addition for a student project.

Warehouse schema (star schema, hosted in ClickHouse + Postgres)

Fact tables (ClickHouse)	Dimension tables (Postgres)
FactUserEvents, FactSessions, FactInteractions, FactConversions, FactBanditDecisions	DimUser, DimProduct, DimPage, DimDevice, DimTime, DimEvent

6. Execution Plan — 13 Build Stages, Fully Detailed

This is one continuous build. Stages are ordered by dependency, not by calendar — move to the next stage only once the current one passes its "Definition of Done." Each stage lists exactly what to build, with which tools, and how you know it's actually finished.

STAGE 0 Environment, Repo & Contracts

Get everyone building against the same ground truth from day one.

- Create the monorepo with folders: /storefront, /tracker-sdk, /ingestion-api, /streaming, /warehouse, /mining, /modeling, /vector-service, /bandit, /llm-copilot, /mlops, /dashboard, /infra.
- Write the canonical event schema (Section 5) and the session-construction rule into a shared /docs/CONTRACTS.md — every teammate reads this before writing a line of code.
- Set up Docker Compose with placeholder services for Kafka, ClickHouse, Postgres, Redis, and Qdrant so everyone can run the whole stack locally with one command.
- Set up GitHub Actions skeleton: lint + unit test job that runs on every pull request from day one.

Definition of Done: any teammate can clone the repo, run one command, and have every backing service (Kafka, ClickHouse, Postgres, Redis, Qdrant) up and reachable.

STAGE 1 Demo Storefront & Tracking SDK

Build the instrument that generates your data.

- Build the Next.js storefront: catalogue, search, product page, reviews, compare, cart, checkout, confirmation. Seed it with ~200 synthetic products across a few categories.
- Write the tracking SDK as a small TypeScript class exposing track(eventType, payload); batch events client-side (flush every 3s or every 10 events) and POST them to the ingestion API.
- Instrument every page/action from the event vocabulary in Section 5.
- Build a lightweight **synthetic traffic generator** (Python script using Playwright or a simple state-machine bot) that simulates hundreds of realistic sessions with configurable convert/abandon/browse probabilities — this is how you get enough data without needing real users.

Definition of Done: running the traffic generator for 10 minutes produces thousands of correctly-shaped events visible in the ingestion API logs, with realistic variety across the three behavior types.

STAGE 2 Real-Time Ingestion & Streaming Backbone

Move events safely, in order, without loss.

- FastAPI endpoint POST /track validates each event against a Pydantic model and publishes it to a Kafka topic user-events, partitioned by session_id (guarantees ordering within a session).
- Add a dead-letter topic user-events-dlq for malformed events instead of silently dropping them.
- Load-test with the traffic generator at increasing rates until you find and document the throughput ceiling of your local setup.

Definition of Done: events published by the tracker appear in the Kafka topic in the correct per-session order, with zero events lost under a sustained burst test.

STAGE 3 Real-Time Processing & the Feature Store

Turn raw events into live, servable features.

- Write a Flink job that consumes user-events, groups by session_id, and maintains rolling session state (event count, time since last event, current funnel stage, running dwell time).
- On each update, Flink writes the current feature vector for that session into Feast's online store (Redis) and appends the historical record to the offline store (Parquet on disk / S3).
- Write a Spark batch job that periodically recomputes heavier aggregate features (e.g. 7-day product popularity, category-level conversion rate) and backfills them into Feast's offline store.

Definition of Done: for any live session_id, a single Feast get_online_features() call returns an up-to-the-second feature vector, and the exact same feature definitions can be pulled historically for model training with no code duplication.

STAGE 4 Warehouse & OLAP Layer

Give every downstream layer one queryable source of truth.

- Design and create the ClickHouse fact tables and Postgres dimension tables from Section 5.
- Build the ETL job (Spark or a scheduled Python script) that lands cleaned, deduplicated events and closed sessions into the warehouse continuously.
- Write and save the 10-15 canonical analytical queries you already know you'll need: funnel drop-off, conversion by segment, product performance, session-length distribution.

Definition of Done: ClickHouse queries over a full day of synthetic traffic return correct, cross-checked numbers (spot-check a handful of sessions by hand against raw Kafka logs).

STAGE 5 Behavioral Data Mining

Discover the patterns before you try to predict them.

- Run PrefixSpan / GSP for frequent sequential patterns (e.g. Search -> Product -> Reviews -> Exit).
- Run FP-Growth for association rules (e.g. two Compare events implies 3x abandonment likelihood).
- Run HDBSCAN (preferred over K-Means here since natural behavioral clusters are rarely spherical or evenly sized) on aggregate session features to discover behavioral personas: fast converters, heavy comparers, explorers, abandoners.
- Document each discovered pattern/cluster with a short plain-English description and a support/confidence or silhouette score.

Definition of Done: a short internal report listing the top 10 sequential patterns, top 10 association rules, and a labeled cluster map, each backed by a concrete metric.

STAGE 6 Sequence Representation Learning (the core research engine)

Teach a model to understand behavior the way a language model understands text.

- Treat each session as a "sentence" whose "words" are events (e.g. product_view_p17, compare_p17, add_to_cart_p17). Build a vocabulary and tokenize every session.
- Implement a compact Transformer encoder (4-6 layers, similar in spirit to SASRec/BERT4Rec) in PyTorch that ingests a tokenized session and outputs a fixed-size embedding.
- Pretrain it **self-supervised** using a contrastive objective: create two augmented views of the same session (randomly mask a few events, crop a sub-sequence, or shuffle a small local window) and train the encoder so the two views land close together in embedding space while different sessions land farther apart (an InfoNCE-style loss, the same family of objective used by SimCLR/CLIP, applied to behavior instead of images/text).
- Only after pretraining, fine-tune a small classification head on top of the frozen/partially-frozen encoder for the labeled intent task (purchase / abandon / browse).
- Keep a simpler baseline encoder (e.g. mean-pooled event embeddings via a pretrained Sentence-Transformer over the stringified sequence) alongside the custom Transformer — you need this baseline for the research comparison in Section 8.

Definition of Done: two embedding spaces exist — a baseline (Sentence-Transformer over stringified sequences) and your custom contrastively-pretrained Transformer — and a quick nearest-neighbor sanity check shows sessions with similar real outcomes landing near each other in both, with the custom encoder visibly tighter.

STAGE 7 Vector Retrieval Engine

Make "find sessions like this one" instant and production-grade.

- Stand up Qdrant, create a collection for session embeddings, and upsert every session's vector with its metadata (outcome, segment, timestamp).
- Expose a small internal API: given a session_id or a live in-progress embedding, return the top-K nearest sessions with their outcomes.
- Add metadata filtering (e.g. "only compare against sessions from the same product category") — this is standard production vector-search practice and elevates the component beyond a bare similarity search.

Definition of Done: querying Qdrant with a known cart-abandoning session returns predominantly other abandoning sessions in its top-10 neighbors, and the query completes in well under 100ms.

STAGE 8 Retrieval-Augmented Intent Prediction & Contextual Bandit Personalization

Turn understanding into decisions.

- Build three intent-prediction variants for the research comparison: (A) traditional aggregate features, (B) hand-crafted sequence/transition features, (C) learned embeddings from Stage 6.
- Build a fourth, novel variant (C+R): take the Stage 7 top-K retrieved neighbors' outcomes and feed them as extra features into the Model C classifier (e.g. "7 of the 10 nearest sessions abandoned") — this is the retrieval-augmented predictor and your headline research contribution.
- Train all four with the same classifier family (start with XGBoost, then try a small Transformer head) so differences come from representation, not model capacity.

- Implement a contextual bandit (Thompson Sampling is easiest to reason about) that, given the current intent prediction, chooses a real-time action: show a discount nudge, show a comparison-help widget, do nothing, etc. Log the action and eventual outcome so the bandit can keep learning online.

Definition of Done: a results table exists comparing A/B/C/C+R on Accuracy, F1, ROC-AUC, and PR-AUC, and the bandit visibly shifts its action distribution over a simulated run as it accumulates reward signal.

STAGE 9 LLM-Powered Natural-Language Analytics Copilot

The showcase feature — make the whole platform conversational.

- Build a text-to-SQL layer: given a natural-language question, an LLM (Claude/GPT via API) generates a ClickHouse query against a documented, restricted schema (only expose approved views, never raw PII-adjacent tables).
- Add a semantic-search tool: for questions like "find sessions similar to users who abandoned after comparing multiple products," the agent calls the Stage 7 Qdrant API instead of SQL.
- Wire both tools into a small agent loop (a manual router, or LangChain/LlamaIndex, or a hand-rolled tool-calling loop against the Anthropic/OpenAI API) that decides which tool a question needs, executes it, and writes a grounded natural-language answer citing the actual numbers returned.
- Add a guardrail step: validate that generated SQL only touches whitelisted tables/columns and enforce the k-anonymity floor from Section 5 before any aggregate is returned.
- Build a simple chat panel in the dashboard so anyone can type: "Which behavioral segment has the highest abandonment this week, and how many sessions is that based on?" and get a real, correct answer.

Definition of Done: the copilot correctly answers at least 15 diverse pre-written test questions spanning simple aggregates, funnel questions, and semantic-similarity questions, with zero hallucinated numbers (every figure is traceable to an actual query result).

STAGE 10 MLOps, Monitoring & Explainability

Prove the system is operable, not just demoable once.

- Log every model training run (hyperparameters, metrics, artifacts) to MLflow; register the best model version as "production" through the MLflow model registry.
- Instrument the ingestion API, Flink job, and prediction service with Prometheus metrics (request rate, latency, error rate, queue lag); build a Grafana dashboard for all of it.
- Set up Evidently AI to compare live incoming feature distributions against the training set weekly (or on-demand) and flag drift.
- Add SHAP to the intent model so every prediction in the dashboard can show "why" as a feature-contribution chart.

Definition of Done: Grafana shows live traffic metrics during a demo run, Evidently produces a drift report on injected synthetic drift (e.g. suddenly more mobile traffic), and SHAP explanations render for arbitrary sample predictions.

STAGE 11 Dashboard & Demo Experience

Make everything above visible and delightful in one place.

- React dashboard: executive overview, live funnel, behavioral segments map, intent-score feed, semantic "find similar sessions" search box, and the embedded LLM chat panel from Stage 9.

- Optional Power BI view for a more traditional stakeholder-style report, generated from the same ClickHouse warehouse.
- Script a 5-minute live demo: start the traffic generator, show live events flowing into Grafana, show an intent score updating in real time for one live session, ask the copilot two or three questions live, then show a SHAP explanation for a prediction.

Definition of Done: the 5-minute demo script runs start-to-finish without manual intervention, on a freshly booted environment.

STAGE 12 Deployment & CI/CD

Package it so it runs anywhere, reproducibly.

- Containerize every service; finalize the single docker-compose.yml that brings up the entire platform.
- GitHub Actions: on every push, run lint + unit tests + a smoke test that boots the compose stack and checks each service's health endpoint.
- Stretch goal: deploy to a single-node Kubernetes cluster (k3s/minikube locally, or a managed cluster on free cloud credits) and provision it with Terraform for full infra-as-code credit.

Definition of Done: a clean machine running only Docker (and optionally kubectl/terraform) can bring the entire platform up from the repo with documented one-line commands.

7. Evaluation Framework & Success Bar for Every Component

Component	Metric(s)	Target to aim for
Streaming pipeline	Event loss rate, end-to-end latency	0% loss under burst; <2s p95 event-to-warehouse latency
Feature store	Online/offline feature parity	100% match on spot-checked features
Sequential/association mining	Support, confidence, lift	Document top patterns with lift > 2.0
Clustering	Silhouette score, Davies-Bouldin index	Silhouette > 0.3 with human-interpretable clusters
Intent prediction (A/B/C/C+R)	Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC	C+R measurably beats A and B; report the exact delta
Semantic retrieval	Precision@K, Recall@K, MRR, NDCG	Precision@10 > 0.7 on held-out sessions
Contextual bandit	Cumulative reward vs. static-rule baseline	Bandit beats a fixed-rule baseline after a defined warm-up window
LLM copilot	Answer correctness on a fixed test question set, hallucination rate	≥90% correct on the test set; 0 unfounded numeric claims
Drift monitoring	Detected vs. injected drift events	100% detection on synthetic drift injections

8. The Research Paper

Title

Retrieval-Augmented Behavioral Intent Prediction: Combining Contrastively Pretrained Session Transformers with Nearest-Neighbor Evidence for Real-Time E-Commerce Decisioning

Research questions

1. Does a contrastively pretrained Transformer session encoder produce embeddings that better separate purchase/abandon/browse outcomes than (a) hand-crafted aggregate features and (b) a generic pretrained sentence-embedding baseline applied to stringified sequences?
2. Does augmenting a session's own embedding with outcome evidence from its top-K nearest historical neighbors (retrieval-augmented prediction) measurably improve intent-prediction accuracy and calibration over embeddings alone?
3. How early in a session (after 3, 5, 10, 20 events) does each representation become reliable enough for real-time use, and does retrieval augmentation help more at early or late session stages?
4. Does a contextual-bandit personalization layer driven by these predictions accumulate measurably more reward than a static, rule-based personalization baseline under simulated traffic?

Experimental design

Experiment	Conditions compared	Metrics
Representation comparison	Model A (aggregate) vs B (sequence stats) vs C (Transformer embedding) vs baseline (generic sentence-embedding)	Accuracy, F1, ROC-AUC, PR-AUC
Retrieval-augmentation ablation	C vs C+R (retrieval-augmented)	Same as above, plus Brier score for calibration
Early-prediction curve	Every representation, measured at session lengths 3/5/10/20 events	F1 and ROC-AUC as a function of session length
Retrieval quality	Qdrant top-K search vs a brute-force cosine baseline	Precision@K, Recall@K, MRR, NDCG, query latency
Decisioning impact	Contextual bandit vs static rule-based targeting	Cumulative simulated reward over time

Why this is a genuine, defensible research contribution

- It isolates *representation quality* from *retrieval augmentation* as two separate, testable variables — most prior undergraduate work conflates them into one "our system vs baseline" comparison.
- The early-prediction curve (RQ3) asks a question standard analytics dashboards never ask, and produces a genuinely new type of chart (accuracy vs. how much of the session has been observed) that reviewers and interviewers immediately find interesting.
- Extending the evaluation into an actual decisioning layer (RQ4, the bandit) goes beyond "we predicted well" into "we showed the prediction improves a downstream decision," which is a much stronger closing argument for a paper.
- The self-supervised contrastive pretraining step mirrors current representation-learning research methodology (the same family as SimCLR/CLIP-style objectives), giving the paper a credible, current methodological anchor to cite and compare against.

Suggested paper structure

1. Introduction & motivation (the two-users-same-duration example makes a strong opening figure)
2. Related work (clickstream analytics, session-based recommendation, contrastive representation learning, retrieval-augmented methods, contextual bandits for personalization)
3. Methodology (event schema, session construction, encoder architecture, contrastive objective, retrieval-augmentation mechanism, bandit formulation)
4. Experimental setup (dataset description — be explicit that traffic is synthetic/simulated plus real collected sessions, evaluation protocol, baselines)
5. Results (all five experiments from the table above, with figures)
6. Discussion (what the early-prediction curve implies for real-time deployment; where retrieval augmentation helps most and why)
7. Limitations & ethics (synthetic data caveats, privacy engineering choices, k-anonymity/differential-privacy notes)
8. Conclusion & future work

9. Execution Checklist (Definition of Done, Master List)

- [ ] Stage 0 — Every teammate can boot the full local stack with one command.
- [ ] Stage 1 — Storefront + tracker + synthetic traffic generator produce realistic, varied sessions.
- [ ] Stage 2 — Zero event loss under burst load; correct per-session ordering in Kafka.
- [ ] Stage 3 — Feast returns live, correct features for any active session; offline/online parity confirmed.
- [ ] Stage 4 — ClickHouse warehouse queries verified against raw logs by hand-spot-check.
- [ ] Stage 5 — Documented top sequential patterns, association rules, and labeled behavioral clusters.
- [ ] Stage 6 — Baseline and contrastively-pretrained Transformer embeddings both exist and pass a nearest-neighbor sanity check.
- [ ] Stage 7 — Qdrant returns correct, fast (<100ms) nearest-neighbor sessions with metadata filtering.
- [ ] Stage 8 — A/B/C/C+R comparison table complete; contextual bandit demonstrably outperforms a static rule baseline.
- [ ] Stage 9 — LLM copilot passes ≥90% of a fixed 15+ question test set with zero hallucinated numbers.
- [ ] Stage 10 — MLflow registry, Grafana dashboards, Evidently drift reports, and SHAP explanations all live.
- [ ] Stage 11 — 5-minute demo script runs end-to-end on a fresh boot without manual intervention.
- [ ] Stage 12 — Entire platform reproducible via Docker Compose (and optionally Kubernetes/Terraform) from a clean machine.
- [ ] Research — All five experiments in Section 8 run, tabulated, and written up with figures.

End of Project Execution Blueprint.